PROJECT REPORT

Project #16

Implementation of Rate-Limited Sender Functionality in picoquic

CUBIC congestion control: baseline, implementation, verification, and experimental evaluation

---

Team members

Aryan Bokolia

Affan Arshad

Soumyadeep Raj

Kevin Shiju George

Aryan Sanjay palimkar

7 November 2026

Baseline revision: `5a4eba475555`
Evaluated implementation: `60323926df8e`

Implementation and experimental evaluation of bounded congestion-window growth for application-limited QUIC connections.

Abstract

This project adds an optional Rate-Limited Increase (RLI) mode to CUBIC in picoquic. The existing CUBIC implementation suppresses window growth when the sender lacks enough application data to fill its congestion window. The new mode permits acknowledgement-driven growth while bounding it using a per-path flight-size history. It implements initialization and reset rules, Slow Start and congestion-avoidance limits, recovery handling, and qlog instrumentation, with an explicit runtime switch for comparison.

The unchanged baseline was rebuilt from commit `5a4eba475555`; the implementation was built from `60323926df8e`. Both passed their complete registered CTest suites on Linux, and the focused `cubic_rli` test passed. Twelve single-flow network-namespace experiments cover two workloads, two versions, and three repetitions per combination. For the rate-limited stream, the mean time-weighted cwnd changed by 98.71%, while the final window rose from 15 KiB to 30 KiB. The application still supplies approximately 192 kbit/s, so this does not imply a doubling of throughput. Mean bulk throughput was 17.732 Mbit/s for baseline and 17.950 Mbit/s for RLI, a 1.23% difference. Six additional experiments run two bulk flows concurrently over a full dumbbell topology. The results demonstrate bounded window growth and characterize the implementation under both isolated and competing traffic.

Contents

Abstract i

1 Project details and objectives 1

2 Background: QUIC, picoquic, and CUBIC 1

2.1 QUIC and picoquic . . . 1

2.2 Quantities used in this report . . . 2

2.3 CUBIC and application limitation . . . 2

3 Baseline definition and observed behavior 2

4 RLI design and relationship to the reference 3

4.1 Why 15 KiB becomes 30 KiB with limited data . . . 3

5 Implementation and changed files 3

5.1 Important implementation decisions . . . 4

5.2 Runtime workflow . . . 4

6 Linux verification and test selection 5

6.1 Complete registered regression suites . . . 5

6.2 Focused RLI behavior checks . . . 5

6.3 Network experiment cases . . . 6

7 Topology and experiment controls **6**

8 Experimental results **8**

8.1 What improved . . . 8

8.2 Bulk-control interpretation . . . 9

8.3 Two competing flows on the full dumbbell . . . 9

9 Graph reading guide and validation verdict **10**

9.1 Limitations and further work . . . 10

10 Conclusion **10**

11 Reproduction and evidence layout **11**

Acknowledgements and tools **11**

A All twelve fresh congestion-window graphs **13**

A.1 Rate-limited, repetition 1 . . . 13

A.2 Rate-limited, repetition 2 . . . 14

A.3 Rate-limited, repetition 3 . . . 15

A.4 Bulk control, repetition 1 . . . 16

A.5 Bulk control, repetition 2 . . . 17

A.6 Bulk control, repetition 3 . . . 18

B Competing-flow dumbbell graphs **19**

B.1 Two-flow dumbbell, repetition 1 . . . 19

B.2 Two-flow dumbbell, repetition 2 . . . 20

B.3 Two-flow dumbbell, repetition 3 . . . 21

1 Project details and objectives

The assignment is to implement rate-limited sender functionality in picoquic, using CUBIC as the congestion control algorithm and Linux network namespaces as the experiment environment. The reference is the IETF CCWG work in progress, *Increase of the Congestion Window when the Sender Is Rate-Limited*, revision 11 [1].

The engineering objectives are to identify how baseline CUBIC handles a sender with insufficient data, implement bounded growth, preserve ordinary congestion responses and opt-out behavior, log the necessary state, and compare observed traces against the intended rules. The report includes the code locations, test evidence, experiment workflow, results, and known limits of the conclusions.

Step	Assignment requirement	Evidence in this project
1	Install picoquic on Linux without a VM	Native Ubuntu builds of both revisions; environment and build logs.
2	Study congestion algorithms and files	Congestion-control study, and file map in Section 5.
3	Create a Linux netns dumbbell topology	Single-flow path and full two-pair dumbbell, with connectivity and qdisc checks.
4	Run baseline CUBIC and plot cwnd	Six new baseline traces: three rate-limited and three bulk.
5	Identify changes for rate limitation	Flight history, ACK growth gate, caps, reductions, and logging design.
6	Implement RLI for CUBIC	Committed implementation and focused regression test.
7	Repeat with RLI and plot cwnd	Six new RLI traces and twelve detailed plots in the report appendix.
8	Validate the expected behavior	Sample-level checks, comparisons, test logs, and documented verdict.

2 Background: QUIC, picoquic, and CUBIC

2.1 QUIC and picoquic

QUIC is a secure transport protocol carried over UDP. It supports multiple streams, connection establishment, and migration between network paths. Its loss recovery and congestion control are specified separately from the core transport [2, 3]. The application produces data, QUIC packages it into packets, and congestion control limits how much may be outstanding while acknowledgements and loss signals update that limit.

picoquic is a C implementation of QUIC with client/server demonstrations, congestion-control callbacks, logging, and test programs [5]. This project changes its userspace CUBIC implementation. Selecting Linux TCP CUBIC with a system-wide TCP setting would not select the congestion algorithm used by a picoquic QUIC connection; the demo's `-G cubic` option does that.

2.2 Quantities used in this report

Quantity	Meaning
`cwnd` / `cwin`	Congestion window: a permitted outstanding-data budget, stored in bytes. The code calls it `cwin`.
FlightSize	Data currently counted as outstanding; picoquic tracks this with `bytes_in_transit`.
`maxFS`	Flight-size history used by RLI, initialized with the initial window.
ACK	Acknowledgement of previously sent data.
RTT	Round-trip time between sending and receiving an acknowledgement.
Throughput	Measured delivered/sent application bytes per second; reported as decimal Mbit/s.
KiB	1,024 bytes. A 15 KiB cwnd is 15,360 bytes; it is a capacity, not a data rate.

2.3 CUBIC and application limitation

CUBIC uses a cubic time-based window target in congestion avoidance and also maintains a Reno-friendly estimate [4]. This picoquic revision contains Slow Start, recovery, HyStart-related behavior, and congestion avoidance in `picoquic/cubic.c`. Its constants and recovery decisions come from that implementation; this report does not assume every detail is identical to Linux TCP CUBIC.

A sender is application-limited when the application provides less data than the permitted congestion window could carry. Increasing cwnd cannot create more application data. A small bytes-in-flight trace below a larger cwnd can therefore be perfectly valid, and a taller cwnd curve alone does not establish a throughput improvement.

3 Baseline definition and observed behavior

The baseline for the new experiments is the unchanged pre-RLI project revision `5a4eba475555`. It is extracted with `git archive` and built separately in `.local-deps/report-baseline/build/`. The two revisions use separately built executables. Binary SHA-256 hashes and exact source commits are recorded with the experiment evidence.

Baseline CUBIC guards normal growth using sender-limited timestamps and, in its Slow Start helper, whether the connection was cwnd-blocked. One relevant gate is:

```
path_x->last_time_acked_data_frame_sent >
    path_x->last_sender_limited_time
```

For the selected low-rate stream, the application does not fill the window, and baseline cwnd remains at its initial value. In this build:

$$\text{initial_cwnd} = 10 \times 1536 = 15360 \text{ bytes} = 15 \text{ KiB}.$$

These constants come from `picoquic/picoquic_internal.h`; 1536 is the compile-time maximum packet-size constant used for the initial window. It should not be confused with the 1200-byte application frame in the workload or the negotiated path MTU.

4 RLI design and relationship to the reference

The draft tracks a flight-history quantity, initializes it to the initial window, updates it as flight size changes, clears it after a window reduction, and bounds subsequent growth [1]. The implementation uses the following sequence:

```
initialize:        maxFS = initial_cwnd
on flight change:  maxFS = max(maxFS, bytes_in_transit)
after reduction:   maxFS = 0
on ACK:           compute candidate congestion window
                  apply an RLI growth limit
                  update pacing and log state
```

The implemented limits are:

$$L_{\mathrm{SS}} = 2\,\mathrm{maxFS}, \qquad L_{\mathrm{CA}} = \mathrm{maxFS} + \mathrm{send_mtu}.$$

Slow Start adds acknowledged bytes before the cap. In congestion avoidance, the implementation computes its CUBIC/Reno candidate and applies the `maxFS + send_mtu` ceiling afterward. Section 3 of the draft gives the additive congestion-avoidance expression; Section 4.5 applies the Section 3 rules to CUBIC while retaining exclusion of application-limited time from the CUBIC epoch [1]. We use the explicit additive expression as a conservative ceiling on the computed candidate. The specification also defines the limit in terms of the algorithm's growth over one acknowledged flight. An alternative time-dependent CUBIC interpretation would require separate derivation and evaluation. The experiments here validate the stated additive-ceiling design, without claiming independent standards certification.

An RLI cap prevents additional growth and does not forcibly shrink a window already above a newly lower ceiling. In simplified form, if a candidate exceeds both the previous window and the current limit, the implementation selects $\max(\mathrm{previous_cwnd}, L)$. Congestion responses and window validation are separate from this growth cap.

4.1 Why 15 KiB becomes 30 KiB with limited data

At connection start, `maxFS` is initialized to 15,360 bytes even if the measured flights are smaller. For these rate-limited runs, it remains at that initial value. ACKs permit bounded Slow Start growth, reaching $2\times15360 = 30720$ bytes, or 30 KiB. Repeated small bursts cannot keep increasing it unless a larger flight is observed.

Thus, `maxFS = 15 KiB` does *not* prove that the application actually sent 15 KiB simultaneously. In these new traces, the largest observed flight is only 3426 bytes. The history includes the initialization floor until a reduction occurs. The connection has a larger permitted window, while the application still supplies only its configured stream. A later burst could use this capacity, but this experiment does not measure a burst-resumption speedup.

5 Implementation and changed files

File	Project change and purpose
`picoquic/picoquic_internal.h`	Adds per-path enable flag, `rli_max_flight_size`, `rli_cwin_limit`, cap flag, and helper declaration.

File	Project change and purpose
`picoquic/sender.c`	Adds `picoquic_update_max_flight_size();` calls it where flight accounting changes, including send and dequeue paths.
`picoquic/cubic.c`	Parses the opt-in switch; initializes and resets state; permits bounded ACK growth; applies SS/CA ceilings; handles recovery and cleanup.
`picoquic/logwriter.c`	Appends RLI state to binary congestion records.
`loglib/qlog.c`	Converts the added binary values into qlog metrics.
`loglib/qlog_fns.c`	Emits RLI metrics in the direct qlog writer.
`picoquictest/congestion_test.c`	Implements the focused `cubic_rli` test and its logical behavior checks.
`picoquictest/picoquictest.h`	Declares the new test.
`picoquic_t/picoquic_t.c`	Registers the named test in the Linux runner.
`UnitTest1/unittest1.cpp`	Registers the test for the Visual Studio harness; that platform is not executed in this verification.
`plot_cwnd.py`	Parses delta qlog metrics with per-path state and plots cwnd, flight size, and RLI state.
`experiments/run_comparison.sh`	Runs baseline/RLI rate-limited and bulk scenarios, saves logs, plots, and metadata.
`experiments/validate_rli.py`	Checks observed growth/reset invariants and produces measured comparisons.
`topology/setup.sh`, `check.sh`, `cleanup.sh`	Single-flow setup, verification, and cleanup.
`topology/setup_dumbbell.sh`, `topology/check_dumbbell.sh`	Full two-pair dumbbell and connectivity checks, added for the competing-flow evaluation.

5.1 Important implementation decisions

Flight accounting observes changes at packet-accounting points so the maximum is not inferred solely from an ACK snapshot that can miss a peak. Following a congestion reduction, later accounting samples rebuild the history. The focused test covers ECN, timeout, accepted loss, and the live send/dequeue paths.

Spurious-loss correction restores CUBIC state without reviving stale pre-reduction flight history. The Reno-friendly shadow window is kept consistent with the permitted cwnd, preventing a stored excessive target from causing later unintended growth. The existing exclusion of application-limited time from the CUBIC epoch is retained and tested.

The feature is opt-in using `-G cubic -H rli=on`. An invocation without the option retains the legacy path. A separate DCUBIC initializer avoids accidentally enabling this CUBIC feature in the shared DCUBIC code. Cap arithmetic saturates at the unsigned integer maximum to avoid overflow; this is a source-level safeguard, not an independently exercised boundary case in the focused test.

5.2 Runtime workflow

1. The application generates the configured media frames or requests a bulk file.

2. picoquic sends packets and updates outstanding bytes and `maxFS`.
3. Incoming ACKs or congestion notifications call the selected CUBIC algorithm.
4. With RLI enabled, CUBIC computes a candidate and applies the applicable growth limit.
5. Loss/ECN/timeout recovery resets history when the window is reduced.
6. qlog and performance CSV records capture the resulting connection behavior.
7. Python reconstructs metric snapshots, validates invariants, and generates plots and comparisons.

6 Linux verification and test selection

Fresh verification ran on native Ubuntu 26.04 LTS, x86-64, kernel 7.0.0-38-generic, GCC 15.2.0, and CMake 4.2.3. The host virtualization check returned `none`. Both versions use RelWithDebInfo builds, the same local picotls source, and the same OpenSSL headers/libraries. The OpenSSL multiarch include directory is supplied explicitly in the baseline build to match the current build configuration.

6.1 Complete registered regression suites

Both registered CTest entries, `picoquic_ct` and `picohttp_ct`, passed for both source revisions. Their internal counts were:

Revision	picoquic subtests	picohttp subtests	Reported failures
Baseline	562	105	0
RLI implementation	563	105	0

The CTest entries use the repository's registered `-n -r` options: debug output is suppressed and failing cases can be retried. The saved logs report zero failures. These are the complete registered default suites, not every possible optional long-running stress/fuzz configuration, external interoperability test, or sanitizer build.

6.2 Focused RLI behavior checks

The single registered `cubic_rli` test includes multiple assertions; they should not be described as separate independently named CTest cases. Its logical coverage includes:

1. Initialization and the initial `maxFS` value.
2. Peak flight-size tracking.
3. Slow Start growth capped at twice the history.
4. Repeated limited bursts retaining the same ceiling.
5. A larger demonstrated flight advancing that ceiling.
6. ECN reduction and history reset.
7. Live send and dequeue flight accounting.

8. Spurious recovery retaining post-reduction history.
9. A cap preventing growth without forcing an additional reduction.
10. Timeout reset and minimum-window response.
11. Isolated loss filtering and accepted-loss reset.
12. Reno-friendly growth below the CA limit.
13. A large CUBIC candidate capped at the CA limit.
14. Application-limited epoch exclusion.
15. Legacy opt-out behavior in Slow Start and congestion avoidance.

6.3 Network experiment cases

The regression suite is not restricted to sender-limited cases. The network evaluation adds two deliberately selected workload classes:

Rate-limited stream: 400 frames of 1200 bytes at 20 frames/s, about 20 seconds and 480,000 application bytes, targeting 192 kbit/s. The quicperf scenario is:

```
=rli:s20:n400:1200:G20:I1200;
```

Bulk control: A 20 MiB file transfer (20,971,520 file bytes), keeping the sender continuously backlogged. The connection CSV can include a small amount of application protocol framing beyond the file size.

Each workload runs three times for baseline and three times for RLI, for twelve runs. Order alternates on repetition two to reduce a simple ordering bias. Three repeats are useful for consistency checks but do not establish statistical significance or coverage of arbitrary paths.

7 Topology and experiment controls

The single-flow comparison uses this four-namespace layout:

```
rls-client - rls-r1 == 20 Mbit/s == rls-r2 - rls-server
```

The client is `10.10.1.2`, the server is `10.10.2.2`, and the router link is `10.10.12.0/30`. Each direction of the bottleneck uses Linux netem with 25 ms delay, a 20 Mbit/s rate, and a 1000-packet queue limit. The base RTT is therefore approximately 50 ms plus processing and queueing.

The scripts ran inside a Linux user/network/mount namespace using `unshare`. A private tmpfs on `/run` provides isolated namespace handles; no guest operating system or VM is involved. The topology check recorded all four namespaces, successful end-to-end pings, and both netem qdiscs. The first ping can include neighbour-resolution delay; the subsequent approximately 50 ms samples reflect the base path more closely than the average of all three samples.

The same topology, certificate files, application parameters, and compiler configuration are used for both revisions. Server-side qlogs are selected because the server is the data sender. Source revisions and demo hashes identify the binaries actually executed.

The supplementary evaluation extends this path to a full dumbbell with two client/server pairs sharing the same inter-router bottleneck:

```
client 1 --- r1 ===== shaped link ===== r2 --- server 1
client 2 ---/                           \--- server 2
```

The second client uses `10.10.3.2`; the second server uses `10.10.4.2`. Both connections start together and each transfers a 20 MiB file. There are six namespaces in total. Three baseline and three RLI repetitions use the same pair of compiled revisions as the original comparison. These supplementary measurements are reported separately, so the original single-flow data is not presented as a competing-flow test.

8 Experimental results

Run	Mode	Final cwnd (KiB)	Avg. cwnd (KiB)	Rate (Mbit/s)	Time (s)	Retrans.
1	baseline	15.00	15.00	0.190	20.189	0
1	rli	30.00	29.81	0.190	20.192	0
2	baseline	15.00	15.00	0.190	20.190	0
2	rli	30.00	29.81	0.190	20.190	0
3	baseline	15.00	15.00	0.190	20.196	0
3	rli	30.00	29.81	0.190	20.190	0

Table 3: Rate-limited workload: fresh measured results. Time is connection duration from the performance CSV.

Run	Mode	Final cwnd (KiB)	Avg. cwnd (KiB)	Rate (Mbit/s)	Time (s)	Retrans.
1	baseline	283.41	201.63	17.731	9.462	0
1	rli	268.04	248.21	17.940	9.352	0
2	baseline	304.68	224.90	17.823	9.413	0
2	rli	261.98	221.30	17.958	9.343	0
3	baseline	295.67	213.74	17.640	9.511	0
3	rli	264.75	233.93	17.952	9.346	0

Table 4: Bulk control workload: fresh measured results. Time is connection duration from the performance CSV.

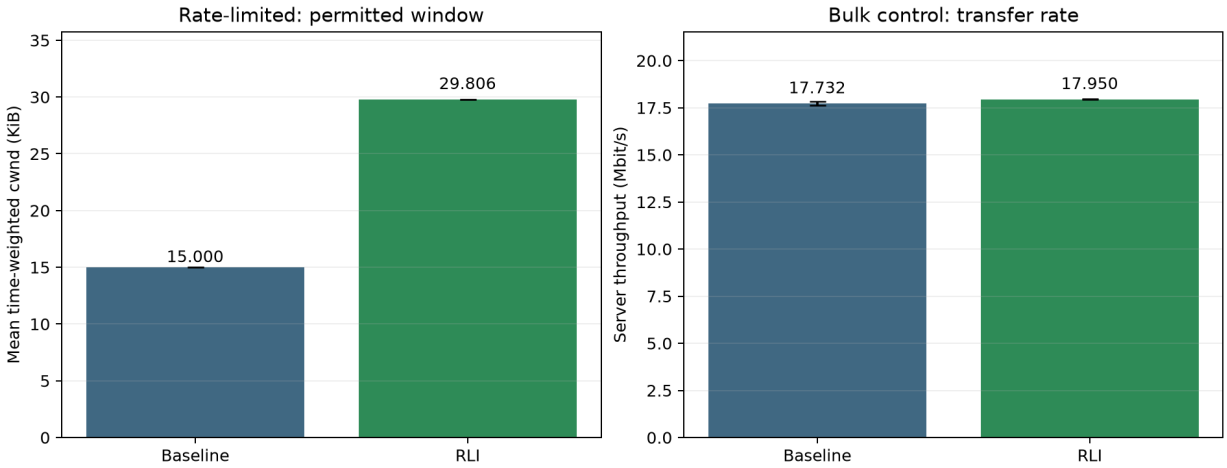


Figure 1: Three-run means with sample standard-deviation error bars. Left: permitted congestion-window behavior under rate limitation. Right: bulk-control throughput. These axes measure different quantities.

8.1 What improved

For the rate-limited workload, baseline mean time-weighted cwnd was 15.000 KiB and RLI mean was 29.806 KiB, an increase of 98.71%. The final RLI window reaches 30 KiB, compared with

15 KiB for baseline, and stays bounded. This demonstrates the intended change from suppressed growth to controlled growth for the tested case.

The mean throughputs were 0.190 and 0.190 Mbit/s. Application supply is the limiting factor, so these measurements do not demonstrate increased useful data delivery. They also do not measure faster restart after a long idle period or a large subsequent application burst.

8.2 Bulk-control interpretation

Mean bulk throughput changed from 17.732 to 17.950 Mbit/s, a 1.23% difference. Small changes between three local runs can arise from scheduling, queueing, ACK timing, and the original CUBIC dynamics. The per-run table and CSV, including retransmission counts, provide the basis for the control comparison. The report does not label a small mean difference as a statistically proven speed gain.

Connection duration in the tables comes from the server performance CSV. The qlog metric span is separately retained in `measurements.csv`. Throughput uses the server's sent-byte count divided by its connection duration, with decimal Mbit/s units. Cwnd statistics use reconstructed metric snapshots and left-step time weighting. The analysis includes the logger's initial zero cap-flag interval when computing cap fractions; the unadjusted generic helper starts that fraction at its first explicit flag observation. This adjustment affects cap-duration statistics, not the cwnd or throughput gains. The final cwnd is state retained at transfer completion, not remaining outstanding data.

8.3 Two competing flows on the full dumbbell

Run	Mode	Flow 1 (Mbit/s)	Flow 2 (Mbit/s)	Combined (Mbit/s)	Jain index	Retrans.
1	baseline	8.737	8.734	18.669	1.000	615
1	rli	8.743	8.781	18.680	1.000	445
2	baseline	9.430	8.776	18.662	0.999	443
2	rli	8.045	7.707	18.574	1.000	477
3	baseline	8.754	8.955	18.665	1.000	292
3	rli	8.753	8.773	18.621	1.000	376

Table 5: Two-flow dumbbell results. Flow rates use each server's connection CSV. Combined rate uses both byte counts divided by the common client wall time, including connection completion.

The mean combined wall-time rates were 18.665 Mbit/s for baseline and 18.625 Mbit/s for RLI (-0.22%). The combined wall interval starts at client launch and ends when both clients exit; it excludes the later wait for server shutdown. The per-flow CSV rates use each server's own connection interval, so their sum is not the combined wall-time rate. The Jain index uses the two per-connection rates; values closer to one indicate a more even split. It describes this pair of flows over their connection lifetimes, not instantaneous fairness or fairness against other congestion algorithms. Both pairs transmit application data through the shared bottleneck. This tests competition between two connections using the same congestion-control mode, rather than competition between different algorithms. The transfer byte counts are checked at both the sender and receiver, and each RLI connection's qlog passes the observed-invariant validator.

9 Graph reading guide and validation verdict

Every detailed graph uses seconds on the x-axis and KiB on the y-axis. Blue is cwnd; grey is bytes in flight. RLI plots also show the flight history, the current growth ceiling, and cap-active markers. Overlapping curves can conceal a limit line when cwnd equals it.

The six rate-limited plots answer whether growth is allowed and bounded when application supply is low. The six bulk plots answer whether a fully loaded connection behaves reasonably with the feature enabled. They should not be compared solely by window height: the workloads deliberately provide very different amounts and timing of data. When a transfer ends, bytes in flight can fall to zero while cwnd remains at its last value.

All six main RLI qlogs pass the observed-invariant validator. The six RLI connections in the supplementary dumbbell runs also pass. For the three rate-limited runs, an active cap is explicitly required; for the bulk runs, the extra check requires a visible cwnd reduction with a flight-history reset. The analysis also checks complete JSON, one sender path, expected transfer byte counts, CSV throughput arithmetic, and baseline traces without RLI metrics.

Verdict: the output is correct for the checked conditions and the implemented limits. Both builds and registered test suites pass, the focused test passes, the rate-limited growth is bounded as intended, and the new experiments support the reported measurements. This is a scoped verification verdict, not a proof of complete IETF conformance or all-network performance improvement.

9.1 Limitations and further work

The experiments use one bottleneck capacity, one base delay, two workloads, and three repetitions per configuration. Additional tests should cover application bursts after idle, receiver flow-control limitation, longer idle periods, competition with different congestion algorithms, multiple paths, variable delay, and deliberately injected loss/ECN. Unit assertions exercise congestion notifications but are not equivalent to all those end-to-end conditions.

The qlog records metric snapshots and can miss intermediate flight peaks; the validator checks what is observable rather than inventing hidden events. The report does not claim that per-path storage itself proves multipath correctness, that overflow boundary cases were independently tested, or that the fixed additive CA ceiling fully resolves the draft's general time-based CUBIC interpretation. Long-term cwnd decay and RFC 7661 window validation remain separate topics. More repetitions and a wider test matrix are needed to quantify performance benefits reliably.

10 Conclusion

The project adds an opt-in RLI mode to picoquic CUBIC and exposes enough state to inspect its operation. The implementation passes the registered regression suites and its focused behavior checks. In the low-rate workload, it permits ACK-driven growth from 15 KiB to a bounded 30 KiB window; the application rate remains essentially unchanged. Bulk and competing-flow experiments provide control measurements under greater offered load. The supported result is controlled window growth with measured behavior under the stated conditions. Further test configurations and independent review are needed before broader conformance or performance claims.

11 Reproduction and evidence layout

Run commands from the repository root. The helper scripts save complete logs and generate tables from raw data instead of manually typed values:

```
cd /home/aryy_02/Desktop/OSNT_Project
python3 "results 1/run_tests.py"
unshare --user --map-root-user --net --mount \
  bash "results 1/run_experiments.sh"
.venv/bin/python "results 1/analyze_results.py"
.venv/bin/python "results 1/analyze_dumbbell.py"
cd "results 1"
../.local-deps/report-tools/tectonic --keep-logs report.tex
pdfinfo report.pdf
pdftotext report.pdf report-extracted.txt
```

The helper assumes the local dependencies documented in the README; a fresh computer should first build picoquic and install the Python plot requirements. Tectonic downloads standard TeX resources on first use. An installed TeX Live distribution can alternatively run `pdflatex report.tex` twice from this folder.

Artifact	Purpose
`report.tex`, `report.pdf`	Editable LaTeX report and compiled output.
`generated-*.tex`	Automatically derived numbers, tables, and graph pages.
`tests/`	Build, CTest, focused-test logs, and detailed CTest records.
`runs/`	All twelve fresh experiment records, qlogs, CSVs, and plots.
`dumbbell/`	Six two-flow experiments, twelve transfers, and supplementary metrics.
`plots/`	Twelve main graphs, one overview, and six competing-flow graphs.
`raw-evidence.tar.gz`	Lossless archive of all raw experiment records.
`comparisons/`, `validation/`	Pairwise CSVs and trace checks.
`environment.json`	Linux/compiler details, commits, and binary hashes.
`measurements.csv`, `summary.json`	Machine-readable measurements and aggregates.
`README.md`	Output correctness assessment and reproduction instructions.
`SHA256SUMS`	Portable file-integrity manifest for the evidence bundle.

Acknowledgements and tools

This work builds on the picoquic library and the referenced IETF specifications. Existing library functionality remains attributable to its contributors. CMake, CTest, Linux network namespaces, Python, matplotlib, and LaTeX were used for building, measurement, and reporting. The conclusions are supported by the archived source revisions and measured outputs described in this report.

References

[1] M. Welzl, T. Henderson, G. Fairhurst, and M. P. Tahiliani. *Increase of the Congestion Window when the Sender Is Rate-Limited*. IETF CCWG, draft-ietf-ccwg-ratelimited-increase-11, September 2026. Work in progress. https://datatracker.ietf.org/doc/html/draft-ietf-ccwg-ratelimited-increase-11

[2] J. Iyengar and M. Thomson. *QUIC: A UDP-Based Multiplexed and Secure Transport*. RFC 9000, May 2021. https://www.rfc-editor.org/rfc/rfc9000.html

[3] J. Iyengar and I. Swett. *QUIC Loss Detection and Congestion Control*. RFC 9002, May 2021. https://www.rfc-editor.org/rfc/rfc9002.html

[4] L. Xu et al. *CUBIC for Fast and Long-Distance Networks*. RFC 9438, August 2023. https://www.rfc-editor.org/rfc/rfc9438.html

[5] Private Octopus contributors. *picoquic source repository*. https://github.com/private-octopus/picoquic Project fork: https://github.com/yoman12357/OSNT_Project

A All twelve fresh congestion-window graphs

Each page below contains a matched baseline/RLI pair from one repetition. The captions give measurements from that exact run. Repetition is used to check consistency, not to imply a different algorithm or test class.

A.1 Rate-limited, repetition 1

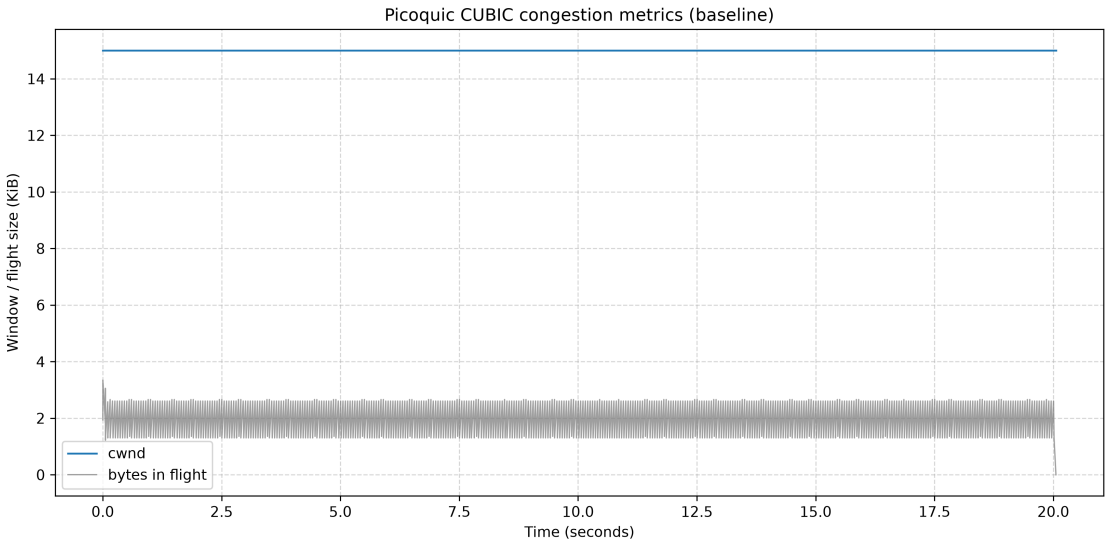


Figure 2: Baseline: final cwnd 15.00 KiB; time-weighted mean 15.00 KiB; throughput 0.190 Mbit/s.

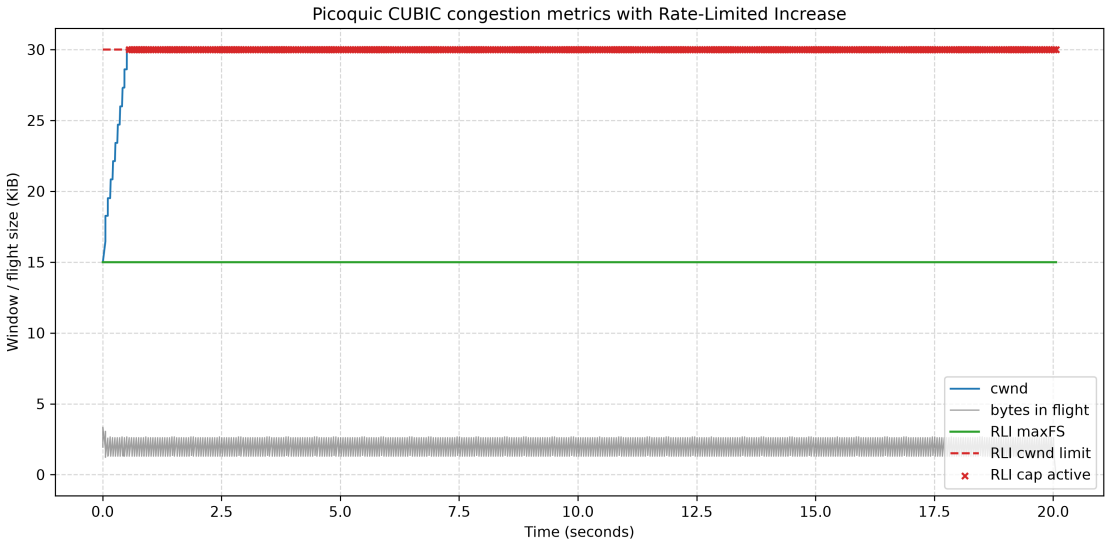


Figure 3: Rli: final cwnd 30.00 KiB; time-weighted mean 29.81 KiB; throughput 0.190 Mbit/s.

These matched traces isolate rate-limited window growth. RLI permits bounded ACK-driven growth; application supply still controls throughput.

A.2 Rate-limited, repetition 2

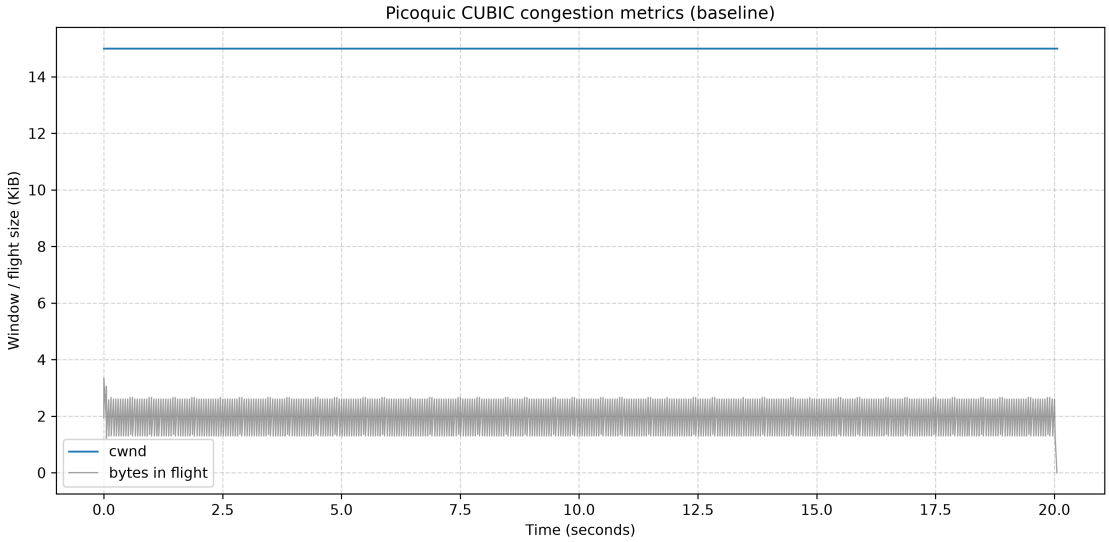


Figure 4: Baseline: final cwnd 15.00 KiB; time-weighted mean 15.00 KiB; throughput 0.190 Mbit/s.

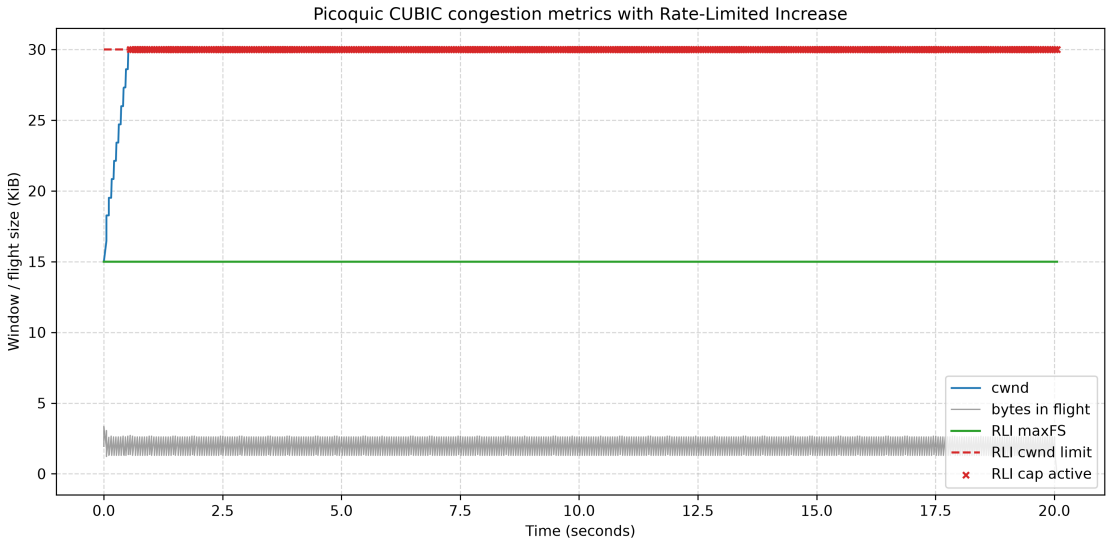


Figure 5: Rli: final cwnd 30.00 KiB; time-weighted mean 29.81 KiB; throughput 0.190 Mbit/s.

These matched traces isolate rate-limited window growth. RLI permits bounded ACK-driven growth; application supply still controls throughput.

A.3 Rate-limited, repetition 3

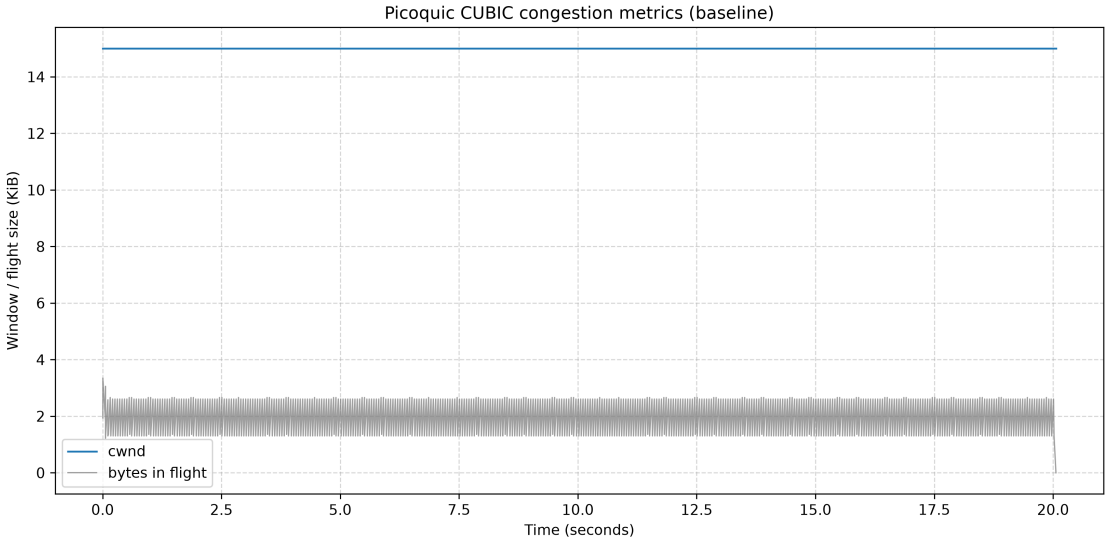


Figure 6: Baseline: final cwnd 15.00 KiB; time-weighted mean 15.00 KiB; throughput 0.190 Mbit/s.

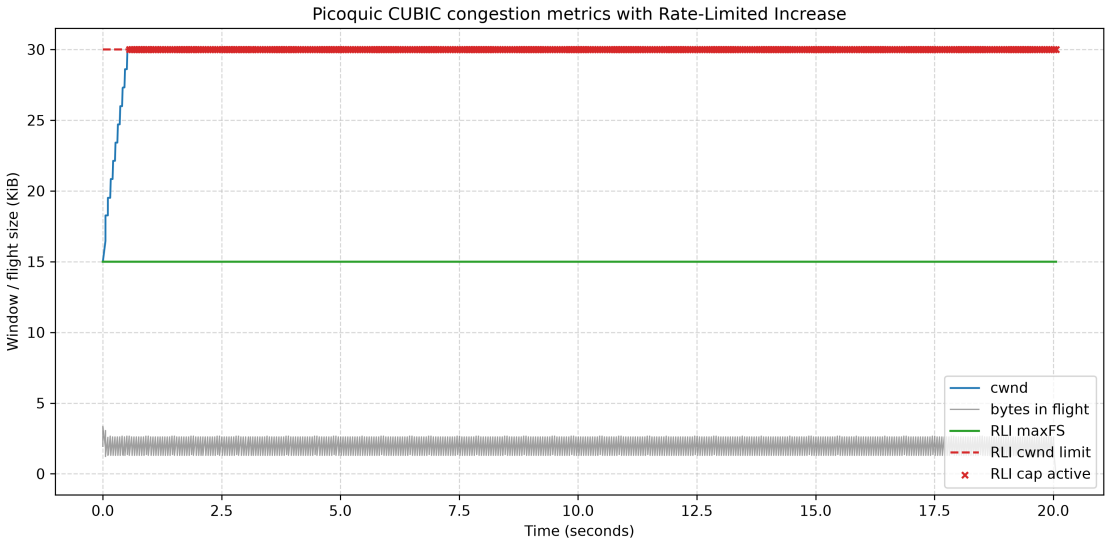


Figure 7: Rli: final cwnd 30.00 KiB; time-weighted mean 29.81 KiB; throughput 0.190 Mbit/s.

These matched traces isolate rate-limited window growth. RLI permits bounded ACK-driven growth; application supply still controls throughput.

A.4 Bulk control, repetition 1

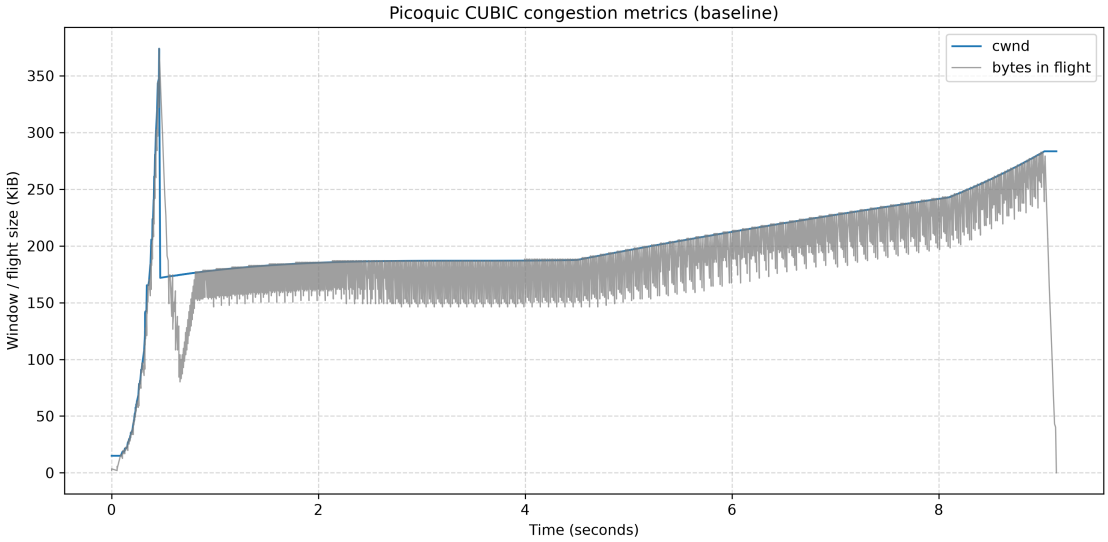


Figure 8: Baseline: final cwnd 283.41 KiB; time-weighted mean 201.63 KiB; throughput 17.731 Mbit/s.

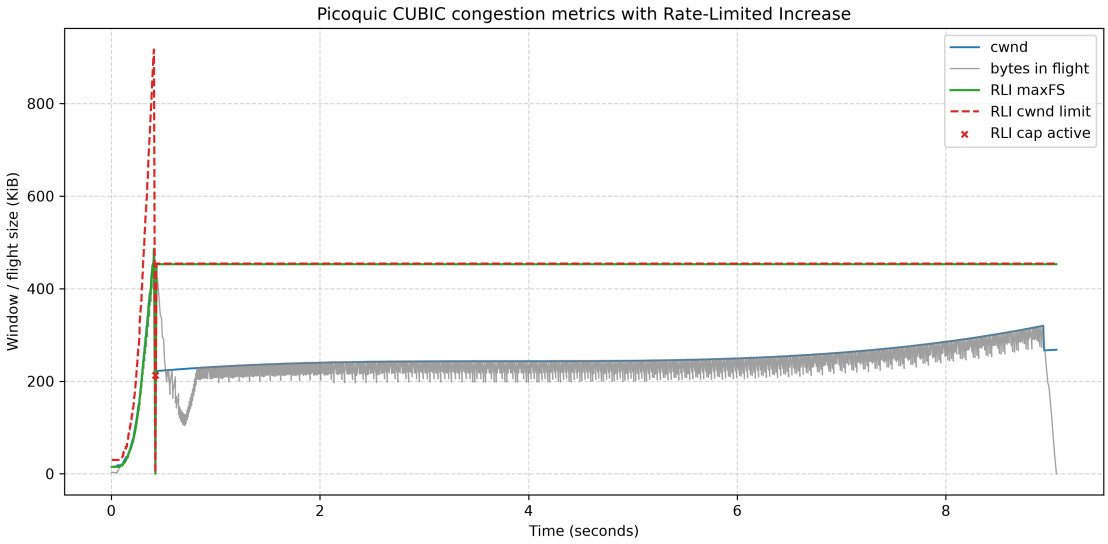


Figure 9: Rli: final cwnd 268.04 KiB; time-weighted mean 248.21 KiB; throughput 17.940 Mbit/s.

These matched traces test a continuously backlogged sender. Normal CUBIC growth and congestion responses dominate; compare measured throughput rather than window height alone.

A.5 Bulk control, repetition 2

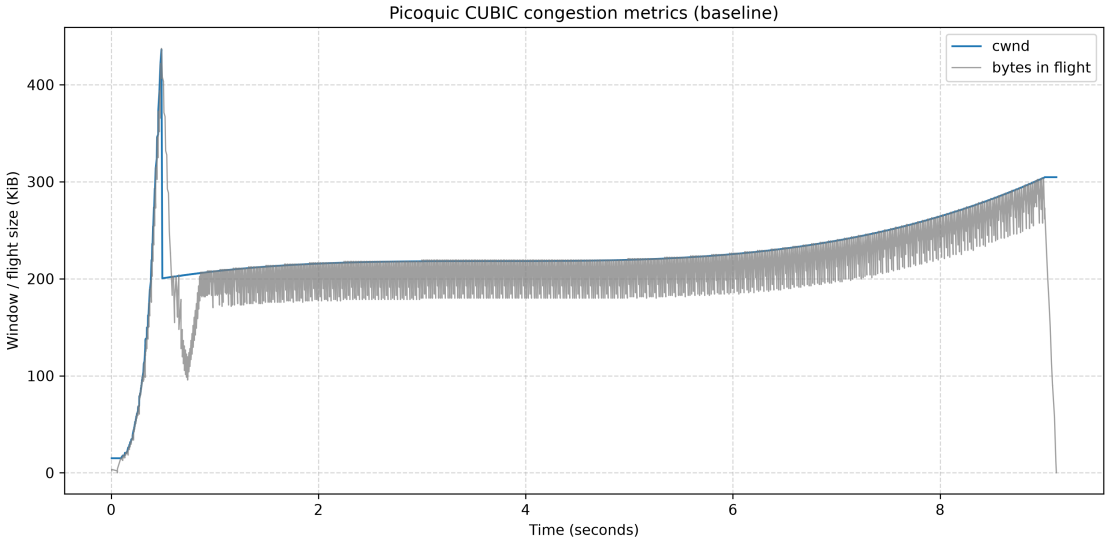


Figure 10: Baseline: final cwnd 304.68 KiB; time-weighted mean 224.90 KiB; throughput 17.823 Mbit/s.

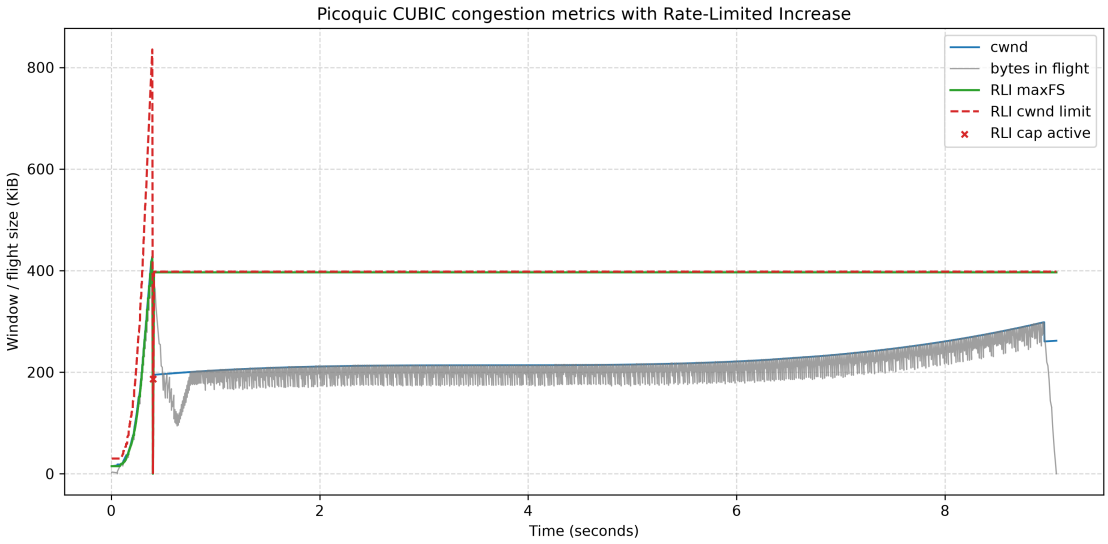


Figure 11: Rli: final cwnd 261.98 KiB; time-weighted mean 221.30 KiB; throughput 17.958 Mbit/s.

These matched traces test a continuously backlogged sender. Normal CUBIC growth and congestion responses dominate; compare measured throughput rather than window height alone.

A.6 Bulk control, repetition 3

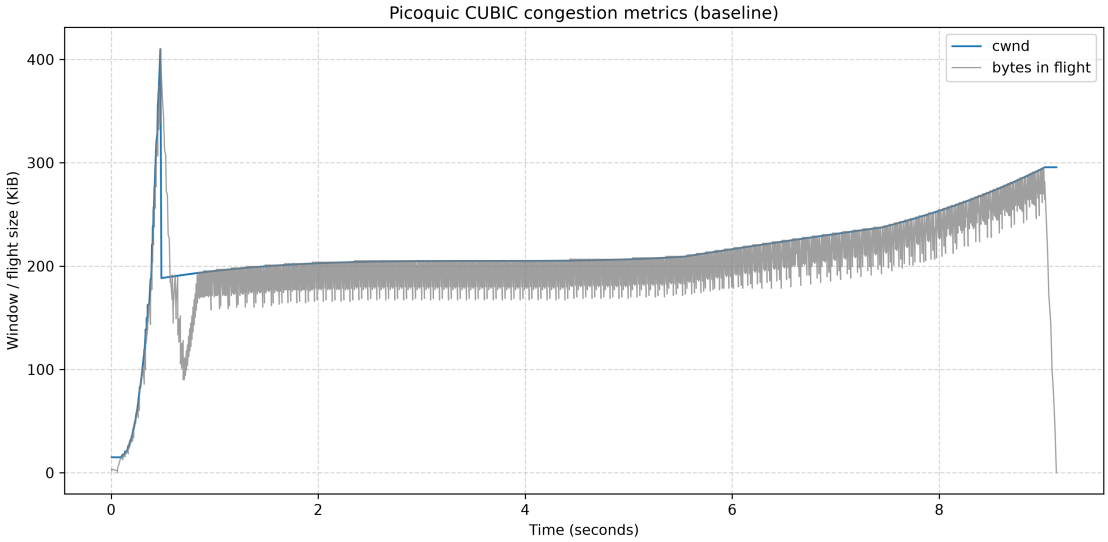


Figure 12: Baseline: final cwnd 295.67 KiB; time-weighted mean 213.74 KiB; throughput 17.640 Mbit/s.

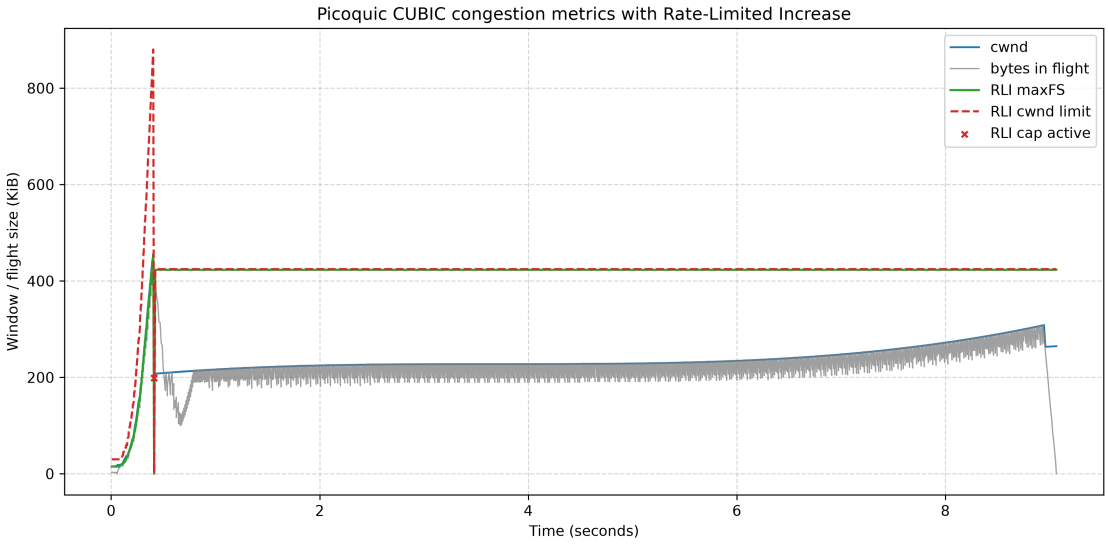


Figure 13: Rli: final cwnd 264.75 KiB; time-weighted mean 233.93 KiB; throughput 17.952 Mbit/s.

These matched traces test a continuously backlogged sender. Normal CUBIC growth and congestion responses dominate; compare measured throughput rather than window height alone.

B Competing-flow dumbbell graphs

Each graph shows two simultaneous 20 MiB transfers over the same bottleneck. The three matched pairs correspond to the supplementary results table.

B.1 Two-flow dumbbell, repetition 1

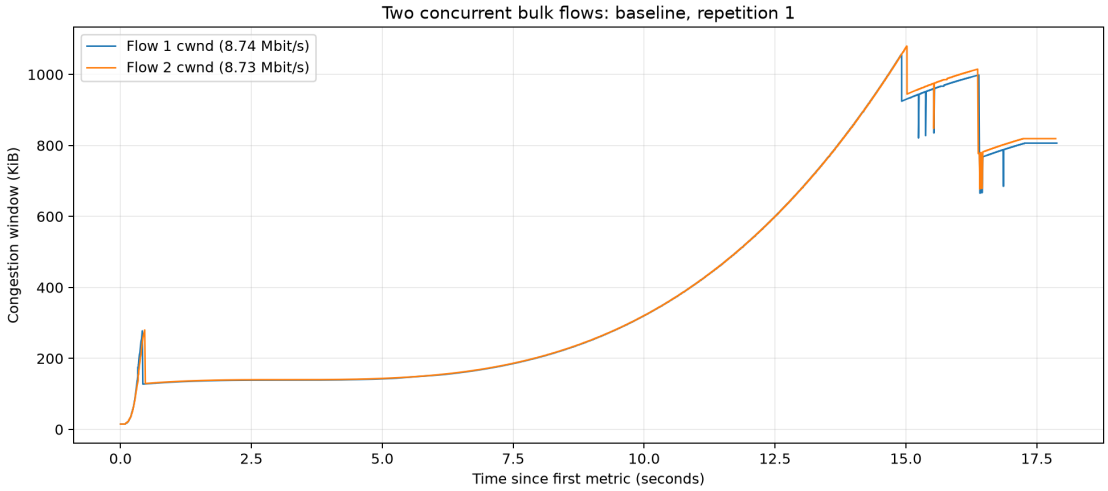


Figure 14: Baseline: both client/server pairs transfer 20 MiB concurrently over the shared bottleneck.

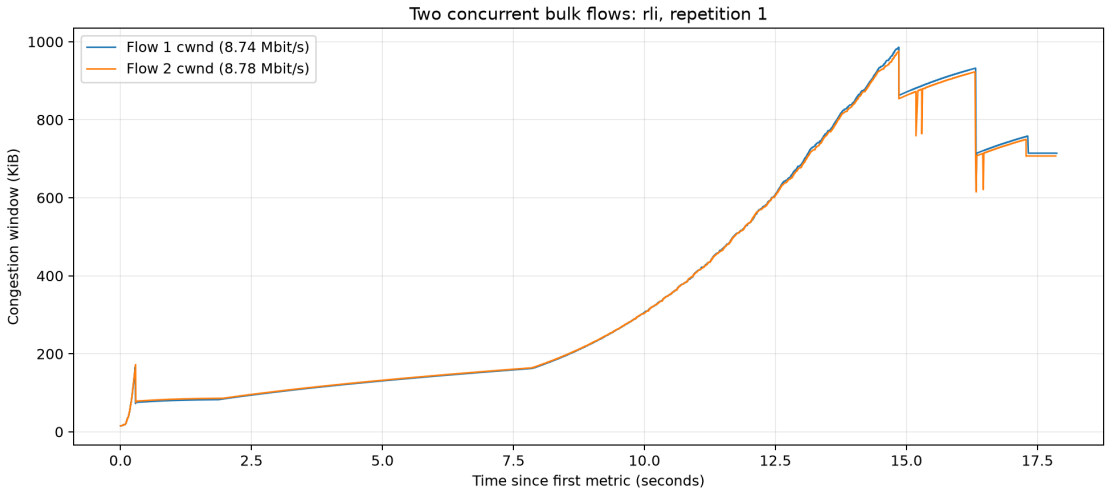


Figure 15: Rli: both client/server pairs transfer 20 MiB concurrently over the shared bottleneck.

The two lines show the congestion windows of competing flows. Window height and temporary differences in allocation should be interpreted alongside the transfer rates and Jain index.

B.2 Two-flow dumbbell, repetition 2

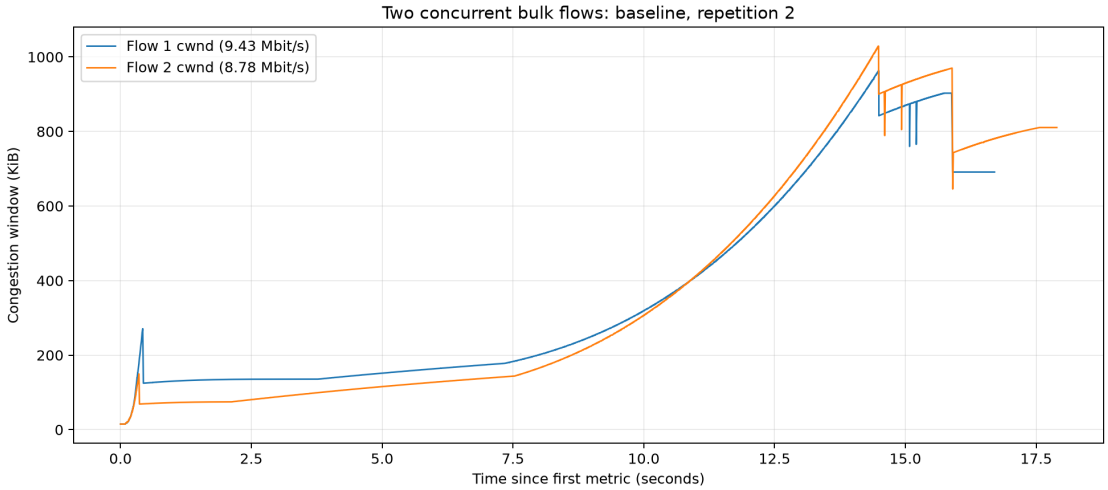


Figure 16: Baseline: both client/server pairs transfer 20 MiB concurrently over the shared bottleneck.

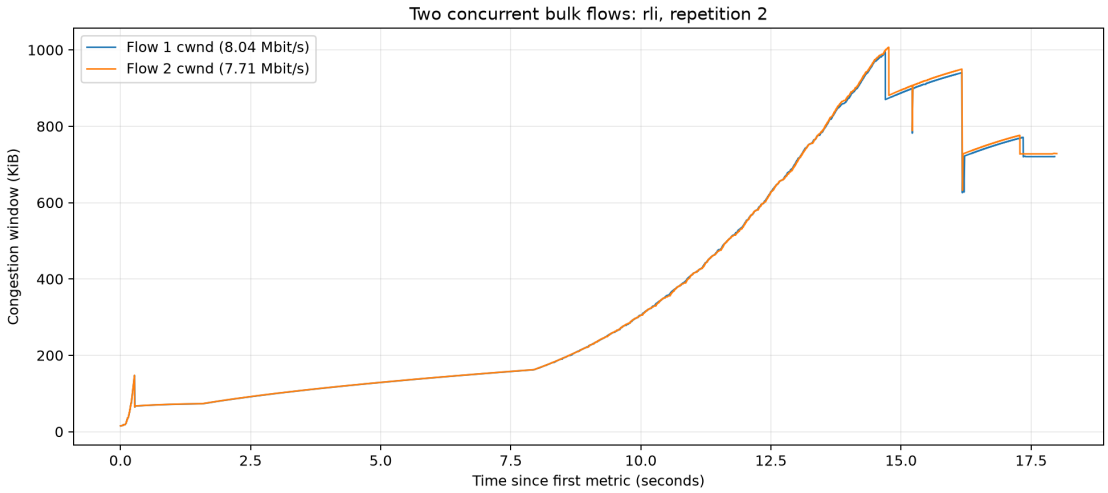


Figure 17: Rli: both client/server pairs transfer 20 MiB concurrently over the shared bottleneck.

The two lines show the congestion windows of competing flows. Window height and temporary differences in allocation should be interpreted alongside the transfer rates and Jain index.

B.3 Two-flow dumbbell, repetition 3

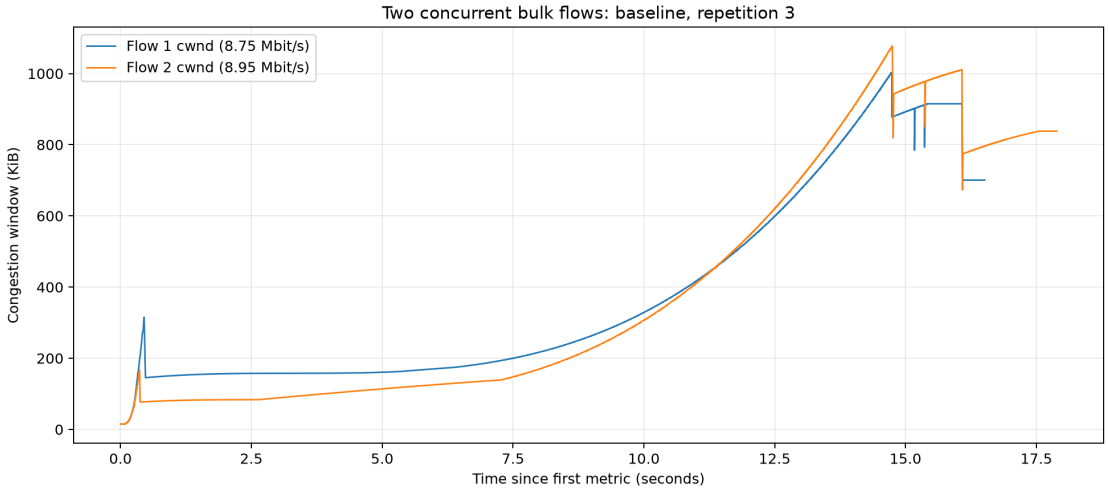


Figure 18: Baseline: both client/server pairs transfer 20 MiB concurrently over the shared bottleneck.

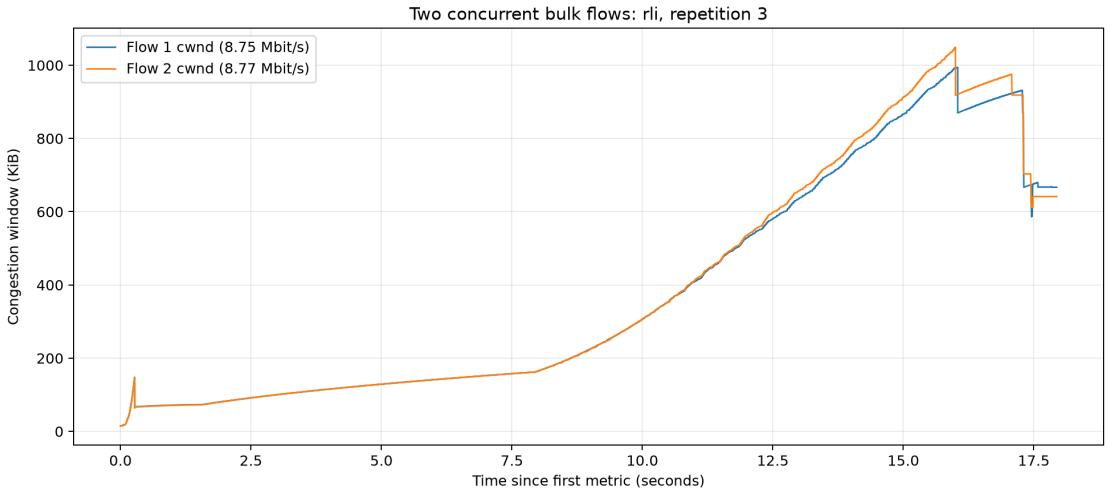


Figure 19: Rli: both client/server pairs transfer 20 MiB concurrently over the shared bottleneck.

The two lines show the congestion windows of competing flows. Window height and temporary differences in allocation should be interpreted alongside the transfer rates and Jain index.